

Lab Work for Week 7: Advanced Models and Queries

Objective

By the end of this lab, students will:

1. Extend Django models with relationships like ForeignKey and ManyToManyField.
2. Write and test advanced queries using Django ORM.
3. Display related data dynamically on templates.

Step-by-Step Lab Tasks

1. Add Advanced Relationships to Models

1. **Task:** Extend the Lesson and Course models to include additional fields and relationships.
 - Update models.py in the courses app:

```
from django.db import models
```

```
class Course(models.Model):
```

```
    title = models.CharField(max_length=200)
```

```
    description = models.TextField()
```

```
    duration = models.IntegerField(help_text="Duration in hours")
```

```
    thumbnail = models.ImageField(upload_to='course_thumbnails/', null=True, blank=True)
```

```
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    def __str__(self):
```

```
        return self.title
```

```
class Lesson(models.Model):
```

```
    course = models.ForeignKey(Course, on_delete=models.CASCADE, related_name='lessons')
```

```
    title = models.CharField(max_length=200)
```

```
    content = models.TextField()
```

```
video_url = models.URLField(null=True, blank=True)

completion_status = models.BooleanField(default=False)
```

```
def __str__(self):
    return self.title
```

2. **Why This Matters:** Adding fields like `video_url` and `completion_status` enhances functionality, while relationships (ForeignKey) enable linking lessons to courses.

2. Track Student Progress

1. **Task:** Add a `ManyToManyField` to track completed lessons for each student.
 - Update the Student model in `models.py`:

```
class Student(models.Model):

    name = models.CharField(max_length=200)

    email = models.EmailField(unique=True)

    enrolled_courses = models.ManyToManyField(Course, related_name='students')

    completed_lessons = models.ManyToManyField(Lesson, related_name='completed_by',
blank=True)
```

```
def __str__(self):
    return self.name
```

2. **Why This Matters:** Tracking completed lessons is essential for building progress tracking features.

3. Apply Migrations

1. **Task:** Generate and apply migrations for the updated models.

```
python manage.py makemigrations
```

```
python manage.py migrate
```

2. **Verify:** Check the database schema to confirm the changes.

4. Query Data Using Django ORM

1. **Task:** Open the Django shell.

```
python manage.py shell
```

2. **Perform Queries:**

- List all courses:

```
from courses.models import Course, Lesson, Student
Course.objects.all()
```

- Get lessons for a specific course:

```
course = Course.objects.get(id=1)
course.lessons.all()
```

- Enroll a student in a course:

```
student = Student.objects.get(email="student@example.com")
course = Course.objects.get(id=1)
student.enrolled_courses.add(course)
```

- Mark a lesson as completed for a student:

```
lesson = Lesson.objects.get(id=1)
student.completed_lessons.add(lesson)
student.completed_lessons.all()
```

- Calculate course progress for a student:

```
total_lessons = course.lessons.count()
completed = student.completed_lessons.filter(course=course).count()
progress = (completed / total_lessons) * 100
print(f"Progress: {progress}%")
```

3. **Why This Matters:** Querying and manipulating data programmatically is essential for building dynamic, data-driven applications.

5. Display Related Data in Templates

1. **Task:** Update the `course_detail.html` template to display lessons and completion status.

- Add the following code:

```

<h1>{{ course.title }}</h1>
<p>{{ course.description }}</p>
<h2>Lessons</h2>
<ul>
    {% for lesson in course.lessons.all %}
        <li>
            {{ lesson.title }}
            {% if lesson in student.completed_lessons.all %}
                (Completed)
            {% else %}
                (Not Completed)
            {% endif %}
        </li>
    {% endfor %}
</ul>

```

2. **Why This Matters:** Templates provide the front-end interface to display dynamic, relational data.

6. Test Progress Tracking

1. **Task:** Log in as a student and enroll in a course.
2. **Task:** Mark lessons as completed via the Django admin interface or shell.
3. **Task:** Visit the course detail page and verify the updated progress display.

Lab Deliverables

1. Updated models.py with advanced relationships.
2. ORM queries tested in the Django shell.
3. Updated course_detail.html displaying lessons and their completion status.
4. Screenshots of:
 - ORM queries and results in the Django shell.

- The course detail page showing lesson progress.

Additional Questions for Students

1. How does the `related_name` parameter affect querying related objects?
2. What are the advantages of using `ManyToManyField` for tracking relationships?
3. How would you calculate progress dynamically in a view instead of the shell?